

Kubernetes CSI & DNS Incident-Response Lab

minikube rehearsal to Azure AKS — provision, break, diagnose, document

D. GRUNWALD · 2026

1 Overview

This lab stands up a small Kubernetes cluster, injects a deliberate CSI-storage failure and a DNS-egress failure, and walks the diagnosis of each using `kubect l`, in-pod Linux inspection, DNS tooling, and packet capture. It runs free on minikube for rehearsal, then once on a trial-friendly Azure AKS cluster to capture evidence backed by real cloud storage.

Sequence: Rehearse on minikube until routine, then run once on AKS and keep that bundle as your portfolio artifact — cloud storage evidence reads far stronger than local hostpath.

2 Performance setup (once)

minikube runs on Docker Desktop's WSL2 VM. The default 2 CPU / 3 GB allocation is too small for a Calico cluster and starves the Docker daemon — the symptom is a docker command hanging 15 seconds and start taking 30+ minutes.

Seen in practice: a two-node Calico cluster wedged Docker Desktop — docker container inspect took 18s and returned context-deadline-exceeded. Fix by reducing load (one node) and raising WSL2 memory.

Fix WSL2 resources

Create `%UserProfile%\wslconfig` — save as `.wslconfig`, not `.wslconfig.txt`:

```
[wsl2]
memory=8GB
processors=4
swap=2GB
```

Apply with `wsl --shutdown`, restart Docker Desktop, then verify:

```
docker info --format "{{.NCPU}}" CPUs, {{.MemTotal}} bytes"
```

One node is enough: use `--nodes 1`. Both faults and the full diagnosis loop behave identically on a single node; a second only adds load.

3 Provision & baseline (minikube)

Calico is mandatory — without a policy-enforcing CNI the DNS fault silently does nothing.

```
python minikube_provision_lab.py --profile irlab --nodes 1 --cni calico
```

Prove the storage path before breaking it. On Windows the redirect needs double quotes, or `cmd.exe` intercepts the `>` before it reaches the pod:

```
kubectl -n irlab exec web-0 -- sh -c "echo lab-ok > ../index.html"
kubectl -n irlab exec web-0 -- cat /usr/share/nginx/html/index.html
```

You want `web-0` Running and PVC `data-web-0` Bound — the CSI provisioner matched your claim to a real volume.

Windows quoting: any command with `>`, `|`, or single quotes behaves differently from `cmd.exe`. Use double quotes, or run inside the pod with `kubectl exec -it web-0 -- sh`.

4 Fault A — CSI storage failure

```
python minikube_inject_fault.py storage
```

A PVC bound to a non-existent StorageClass, plus a consumer pod. Diagnose:

```
kubectl -n irlab get pvc                # broken-data = Pending
kubectl -n irlab describe pvc broken-data # Events = the cause
kubectl -n irlab describe pod broken-consumer
```

The describe Events carry the truth:

```
ProvisioningFailed persistentvolume-controller
storageclass "nonexistent-csi-class" not found
```

Causal chain: missing StorageClass → PVC can't provision (Pending) → dependent pod can't get its volume → scheduler refuses to place it (FailedScheduling) → pod Pending. One misconfiguration, three components, two commands.

Transferable habit: `get` shows the symptom; `describe Events` shows the cause. Always read Events for the reason. Identical on AKS — only the message changes.

Capture while broken, then revert:

```
python minikube_collect_evidence.py --out ./evidence
python minikube_inject_fault.py --revert storage
```

Where AKS differs — the important one

Layer	minikube signal	AKS signal (richer)
PVC	Pending — no such StorageClass	Pending/Bound but volume won't attach
Pod events	FailedScheduling / no class	FailedAttachVolume with Azure API detail
Controller logs	n/a — hostpath is local	AttachVolume call + SKU/zone/quota error
Node evidence	df / mount only	plus dmesg SCSI/NVMe attach lines

The interview line: 'I read the csi-azuredisk-controller logs and found the AttachVolume API error.' That log is the real root cause and has no minikube equivalent.

5 Fault B — DNS egress block

A NetworkPolicy allowing egress on 80/443 but omitting port 53. The pod keeps Running while name resolution breaks. Transfers 1:1 to AKS.

```
python minikube_inject_fault.py network
```

The nginx pod has no nslookup: kubectl exec web-0 -- nslookup returns 'executable not found' (exit 127) — a missing tool, not the DNS block. Exit 127 is not-found, not a timeout.

Reproduce correctly with the built-in verifier (runs the lookup from busybox:1.28):

```
python minikube_inject_fault.py network --verify
```

Two lookups: [1] from an app=web pod (policy applies → times out) and [2] from an unlabeled pod (policy does not apply → resolves). [1] failing while [2] succeeds proves the block is policy-scoped, not a broken resolver.

Confirm the resolver is sane, then name the culprit:

```
kubectl -n irlab exec web-0 -- cat /etc/resolv.conf
kubectl -n irlab describe networkpolicy deny-dns-egress
```

Controlled comparison: the unlabeled pod resolving while app=web times out isolates the variable perfectly — same cluster, same DNS, only the policy-selected pod fails.

6 Graduating to AKS

Dimension	Minikube (rehearsal)	AKS (real run)
Nodes	Local containers, no quota	Real Azure VMs, vCPU quota
Storage	hostpath — local, no attach	Managed disk — real CSI attach
Storage fault	Missing class -> Pending	Bad SKU -> FailedAttachVolume
Root evidence	Pod/PVC events	Controller logs: AttachVolume + error
Network / DNS	Calico enforces policy	Azure CNI enforces — 1:1
Cost	Free	Node VM + disk; delete when done

Trial quota: Free Trial cannot raise vCPU quota. One Standard_B2s node (2 vCPU) fits. If create fails, try another region or upgrade to pay-as-you-go.

7 Corrections from the run

Symptom	Cause	Fix in scripts
docker hung 18s; 30+ min start	WSL2 starved, 2-node Calico	1 node default; .wslconfig 4/8
'path not found' on seed	cmd.exe ate the > redirect	double-quoted seed command
nslookup exit 127	nginx has no nslookup	--verify uses busybox:1.28
describe lacked Events	describe truncation	collector saves PVC as YAML

© 2026 David Grunwald. All rights reserved.